

Institute of Science & Technology

Department of Computer Science and Engineering

Result Processing System

Role Based Academic Result Management

A project report submitted in partial fulfillment of the requirements
for the degree of Bachelor of Computer Science and Engineering

Submitted By

Kazi Saif Hossain Emon

Registration No 21502005399

Roll No 22075

Session 2021–2022

Supervised By

Saleh Muhammad Maruf

Assistant Professor

Department of Computer Science and Engineering

Date of Submission 4 November 2026

ABSTRACT

Academic result preparation requires consistent course registration, controlled marks entry and a clear decision about when results become visible to students. The Result Processing System (RPS) addresses these needs through a role-based web application that connects academic records, course assignments, assessment marks and result publication in a single local workflow. The objective is to reduce repeated data handling and prevent common errors involving batch identity, unauthorized access and premature result disclosure.

The application is implemented with FastAPI, SQLAlchemy and SQLite, supported by an HTML, CSS and JavaScript interface. Administrators maintain academic data and accounts, assign teachers to session-specific courses, configure batch semesters and publish reviewed results. Teachers enter four validated assessment components within their assigned scope. Students view their own published results, including credit-weighted semester GPA and CGPA. Fixed core-course enrollment, CSV import validation, revision checks and publication locks support data consistency.

Evaluation in this report uses isolated unit tests of grading functions, marks validation and frontend state handlers. Twelve Python test methods and six JavaScript test methods passed. Interface snapshots illustrate the implemented role-specific workspaces using demonstration records. The project provides a practical local academic result-processing prototype. Institution-approved grading policies, retakes, audit history and public deployment remain areas for further development.

Keywords: Result Processing System, academic records, role-based access control, result publication, GPA, CGPA.

DECLARATION

I, Kazi Saif Hossain Emon, Registration No 21502005399, Roll No 22075, Session 2021–2022, of the Department of Computer Science and Engineering at the Institute of Science & Technology, declare that this project report titled “Result Processing System” presents the software project prepared for my academic submission under the supervision of Saleh Muhammad Maruf, Assistant Professor.

The report explains the requirements, design, implementation and unit testing of the project. Technical ideas and documentation used in preparing the report are acknowledged through the references. The project is submitted for academic evaluation; demonstration results shown in the report are not official academic records.

Signature _____

Kazi Saif Hossain Emon
Registration No 21502005399
Roll No 22075
Session 2021–2022

Date 4 November 2026

ACKNOWLEDGEMENT

I express my gratitude to Almighty Allah for the strength and opportunity to complete this project and report.

I sincerely thank my supervisor, Saleh Muhammad Maruf, Assistant Professor in the Department of Computer Science and Engineering at the Institute of Science & Technology, for academic guidance and supervision of this work.

I am grateful to the teachers and staff of the department for the learning environment and support provided during my studies. I also thank my classmates for their encouragement and the project report shared as a reference for the organization of this book.

I acknowledge the developers and maintainers of the open-source technologies and technical documentation used in the Result Processing System. Finally, I thank my parents and family for their patience, encouragement and support throughout my education.

Kazi Saif Hossain Emon

CONTENTS

Abstract.....i
Declaration.....ii
Acknowledgement.....iii
Chapter 1 Introduction.....1
1.1 Definition of the system.....1
1.2 Problem statement.....1
1.3 Purpose.....1
1.4 Report organization.....1
1.5 Objectives.....2
1.6 Scope of the implemented system.....2
1.7 Project boundaries.....2
1.8 Academic batch configuration.....2
Chapter 2 Literature review.....3
2.1 Academic result-processing approaches.....3
2.2 Role-based access control.....3
2.3 Authentication and validation.....3
2.4 Relational integrity and transactions.....4
2.5 Optimistic concurrency.....4
2.6 Scope comparison.....4
Chapter 3 Feasibility study.....5
3.1 Operational feasibility.....5
3.2 Technical feasibility.....5
3.3 Economic feasibility.....5
3.4 Adoption and schedule feasibility.....5
Chapter 4 Requirement analysis.....6
4.1 Development methodology.....6
4.2 Actors and responsibilities.....6
4.3 Functional requirements.....6
4.4 Core business rules.....7
4.5 Privacy and execution environment.....7
Chapter 5 System design.....8

CONTENTS CONTINUED

5.1 Architecture and input design....8
5.2 Data flow and transactions....8
5.3 Use case diagram....9
5.4 Entity relationship diagram....10
5.5 Sequence diagram....11
Chapter 6 Implementation and results....12
6.1 Implementation structure....12
6.2 Account and database lifecycle....12
6.3 Assessment and grade calculation....12
6.4 GPA and CGPA....12
6.5 Login and administrative dashboard....13
6.6 Teacher entry and administrative review....14
6.7 Student results and implementation outcome....15
6.8 Operational boundaries....15
Chapter 7 Testing....16
7.1 Unit testing method and isolation....16
7.2 Unit cases and outcomes....16
7.3 Frontend cases and execution snapshot....17
7.4 Interpretation....17
Chapter 8 Conclusions and future work....18
8.1 Conclusion....18
8.2 Limitations....18
8.3 Future development....18
8.4 Completion status....18
References....19

LIST OF FIGURES

Figure 5.1 Layered architecture of the Result Processing System....8
Figure 5.2 Main academic data flows and logical data stores....8
Figure 5.3 Role-specific use cases....9
Figure 5.4 Database entities and principal relationships....10
Figure 5.5 Save review publish and view sequence....11
Figure 6.1 Redesigned login interface....13
Figure 6.2 Administrative dashboard with batch progress....13
Figure 6.3 Teacher marks entry in an assigned session....14
Figure 6.4 Component-level result review before publication....14
Figure 6.5 Student result view with verified GPA and CGPA....15
Figure 7.1 Captured Python and frontend unit test output....17

LIST OF TABLES

Table 2.1 Comparison of result-processing approaches....4
Table 3.1 Development milestones....5
Table 4.1 Functional requirements....6
Table 4.2 Nonfunctional requirements....7
Table 4.3 Execution environment....7
Table 6.1 Implemented grade thresholds....12
Table 7.1 Executed unit test summary....16
Table 7.2 Backend unit case outcomes....16

CHAPTER 1
INTRODUCTION

1.1 Definition of the system

The Result Processing System (RPS) is a role-based web application for organizing academic records and processing semester results. It brings department and session records, students, teachers, curriculum courses, teaching assignments, assessment marks, publication and student result viewing into one local browser-based workspace. The application is developed using FastAPI, SQLAlchemy and SQLite, with a plain HTML, CSS and JavaScript interface.

1.2 Problem statement

A result-processing workflow involves more than adding marks. Each result must belong to the correct student, course, department, academic session and semester. When separate registers or spreadsheets are maintained without a shared workflow, repeated entry, inconsistent course lists and accidental access to the wrong batch become possible. A teacher also needs a clear boundary between the courses they teach and the records they are permitted to change.

A second problem is the distinction between a student's current semester and the semester whose results are being processed. The prepared CSE 2021–2022 batch is currently in Semester 7, while the intended demonstration processes Semester 6 results. Treating these two values as the same would make past-semester result processing unnecessarily difficult.

1.3 Purpose

The purpose of RPS is to provide a small, understandable academic application in which administrators control academic setup, teachers enter marks within an assigned scope, and students view their own published results. It supports the practical sequence from registering a batch to reviewing and publishing a result, while preserving a clear separation between draft work and student-visible information.

1.4 Report organization

This report presents the project background, relevant technical concepts, feasibility, requirements, design, implementation and test evidence. Chapter 7 reports isolated unit tests of grading, input validation and frontend state. Chapter 8 discusses limitations and future work. The references identify the technical sources used throughout the book.

Objectives and scope

1.5 Objectives

- Maintain department, session, student, teacher and course records in a consistent relational database.
- Automatically assign the department’s fixed core curriculum to students of a configured academic batch.
- Assign teachers to courses for a specific session and prevent access to other sessions.
- Validate assessment marks, calculate totals and grades, and derive credit-weighted semester GPA and CGPA.
- Separate draft marks from published results and protect saved records against stale edits.
- Provide a clear user interface, a reproducible startup procedure, database backup and a practical user manual.

1.6 Scope of the implemented system

The implemented local application includes account provisioning, temporary passwords, forced first-login password change, role-based access checks, CSV imports for students and teachers, batch settings, fixed-course enrollment, teacher assignment, marks entry and result publication. The prepared curriculum contains 81 CSE courses: 53 core courses, 14 optional theory courses and 14 optional lab courses. Optional choices are made separately in Semester 8.

1.7 Project boundaries

RPS is a local academic demonstration rather than an institution-approved production service. It does not implement retake attempts, replacement of previous grades, automatic email delivery, single sign-on, printable official transcripts or a complete audit-history module. It does not claim that its configured grading policy has been formally approved by the institution. Public hosting requires additional deployment, security, backup and operational review.

1.8 Academic batch configuration

A batch is identified by its department and academic session. Its current semester determines which marks may be entered, while its processing semester selects the results being prepared. For the CSE 2021-2022 configuration, these values are 7 and 6 respectively. Keeping them separate supports the common situation in which teaching has moved forward while an earlier semester is still being processed.

CHAPTER 2
LITERATURE REVIEW

2.1 Academic result-processing approaches

Academic results can be maintained using paper registers, spreadsheet workbooks or a structured application. These approaches differ mainly in how they represent relationships, validate inputs, separate responsibilities and control publication. A spreadsheet can calculate a GPA correctly when its formulas and data are maintained carefully; the challenge addressed by RPS is the surrounding workflow, including batch identity, teaching responsibility, access control and review status.

RPS therefore models the academic entities explicitly. A student belongs to a department and session, a course belongs to a department and semester, and a teaching assignment joins a teacher, course and session. This design makes the batch boundary a condition checked by the application rather than a convention stored in a worksheet name.

2.2 Role-based access control

Role-based access control associates permissions with organizational responsibilities. NIST describes RBAC as restricting access according to user roles [3]. RPS applies this concept through three roles: administrator, teacher and student. Role checks alone are not sufficient: a teacher must also match the assigned teacher code, course and session, while a student request must match that student’s reference code.

2.3 Authentication and validation

FastAPI documents password hashing, bearer-token authentication and JWT validation as components of an authenticated application [1]. RPS uses this general approach with password hashes, expiring tokens and database-backed user checks. Input validation is performed on the server as well as in the interface. Browser controls improve usability, but the server remains responsible for rejecting invalid marks, duplicate records and unauthorized changes.

Technical foundations and comparison

2.4 Relational integrity and transactions

A relational database can express dependencies between academic records. SQLite documents foreign keys as a way to enforce that referenced records exist; the feature must be enabled for each connection [2]. RPS enables foreign-key enforcement and groups related writes within transactions. For example, a batch import is validated before insertion so that invalid rows do not leave a partially imported batch.

2.5 Optimistic concurrency

A user may review data that another user subsequently changes. SQLAlchemy's documentation discusses version counters for detecting stale records [4]. RPS implements its own revision-checked conditional SQL updates for marks and publication. It does not configure the SQLAlchemy mapper version counter. The browser submits the revision it displayed; a mismatch returns a conflict instead of silently replacing a newer result.

2.6 Scope comparison

Table 2.1 Comparison of result-processing approaches

Criterion	Separate register or workbook	Implemented RPS
Academic relationships	Maintained through conventions and links	Department, session, course and enrollment records
Teacher access	Depends on document-sharing arrangements	Role plus course and session assignment checks
Publication	Usually a separate distribution step	Explicit draft and published states
Concurrent edits	Depends on the document platform	Marks revision checked before save or publish
Deployment	Document or spreadsheet environment	Local Python application with SQLite
Institutional fit	Highly adaptable manually	Fixed project rules requiring institutional review

The comparison describes the operational scope of this project, not a measured benchmark of commercial products. RPS concentrates on a small, demonstrable workflow and avoids claiming the reporting, scalability or integration capabilities of a full university information system.

CHAPTER 3

FEASIBILITY STUDY

3.1 Operational feasibility

The three roles correspond to familiar academic responsibilities. Administrators prepare records and review results; teachers enter assessment marks; students read published results. Fixed curriculum enrollment removes repeated course assignment for every student. Separate current and processing semesters allow the prepared batch to study Semester 7 while Semester 6 results are processed.

3.2 Technical feasibility

The application runs from one local address and serves both the interface and API. SQLite avoids a separate database-server installation for the demonstration. The backend uses typed validation, relational records and transaction boundaries. No controlled load benchmark has been performed; suitability for a larger workload requires measurement.

3.3 Economic feasibility

The project relies on open-source development tools and can be demonstrated on an existing computer. The principal practical costs are development time, data preparation, testing and maintenance. Public hosting would introduce additional operational costs, including a hosting environment, backups, secure access and administration. The feasibility assessment is qualitative; institutional deployment would require a formal cost and benefit review.

3.4 Adoption and schedule feasibility

The interface uses ordinary forms and role-specific navigation. Name-and-code selectors reduce the need to memorize identifiers, and temporary-password messages support first-time users. Development proceeded through repeated requirement clarification, implementation and testing. The milestones below summarize the implemented development phases.

Table 3.1 Development milestones

Milestone	Evidence
Academic model and setup	Application models and startup commands
Role workflow	Separate admin teacher and student workspaces
Calculation and validation checks	Twelve isolated Python unit methods
Frontend state checks	Six isolated JavaScript unit methods
Documentation	Eight chapters with diagrams and interface snapshots

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Development methodology

RPS was developed iteratively: academic models and role workspaces were followed by session-specific assignments, fixed enrollment, marks safeguards and interface refinement. Chapter 7 reports isolated unit checks without using the real academic database.

4.2 Actors and responsibilities

Administrators manage records, accounts, assignments and publication. Teachers save draft marks only for assigned courses and sessions. Students view only their own published results. Every role can change its own password.

4.3 Functional requirements

The requirements below define the implemented academic workflow and its acceptance conditions.

Table 4.1 Functional requirements

ID	Requirement	Acceptance condition
FR01	Authenticate users	Valid credentials return an expiring token
FR02	Provision accounts	Temporary password displayed after creation/reset
FR03	Maintain academic registries	Validated create, edit and protected delete
FR04	Import students and teachers	Preview first; invalid batch writes nothing
FR05	Configure academic batches	Processing semester does not exceed current semester
FR06	Apply fixed curriculum	Core enrollment repeats without duplicates
FR07	Assign teachers	Assignment includes teacher, course and session
FR08	Save assessment marks	Four bounded values and revision checks
FR09	Publish reviewed results	Only admin can change publication state
FR10	Calculate results	Credit-weighted GPA/CGPA with pending exclusions
FR11	Manage Semester 8 electives	At most one optional theory and one lab
FR12	Back up local data	SQLite backup saved without changing records

Quality requirements and business rules

Table 4.2 Nonfunctional requirements

Quality	Implemented measure
Security	Role/ownership checks, password hashes and token expiration
Integrity	Foreign keys, unique constraints and transaction boundaries
Usability	Role-specific navigation, search and clear confirmation controls
Reliability	Conflicts rejected instead of overwriting newer marks
Maintainability	Separated models, database setup, service logic and tests
Portability	Local browser interface and documented Python startup
Recoverability	Backup command and manual restoration procedure

4.4 Core business rules

A batch is identified by department and session. Its processing semester cannot exceed its current semester. Fixed core enrollment is automatic; Semester 8 electives are selected separately. Teachers cannot edit published marks or enter future-semester marks. Assessment maxima are 10, 20, 40 and 80, with at most two decimal places. Saved results protect credits, course type and linked enrollments from inconsistent changes.

4.5 Privacy and execution environment

Account lists exclude password hashes. Server checks enforce student ownership and teacher course/session scope. Screenshots use fictional records. The portal runs locally at 127.0.0.1:8001, using Python and the pinned dependencies below; a modern browser and available disk space are required.

Table 4.3 Execution environment

Component	Project configuration
Backend	Python, FastAPI 0.141.1 and Uvicorn 0.53.0
Persistence	SQLAlchemy 2.1.4 and SQLite
Validation and security	Pydantic 2.13.5, PyJWT 2.15.1 and pwdlib 0.3.1
Frontend	HTML, CSS and JavaScript without a frontend build step
Local address	http://127.0.0.1:8001
Practical equipment	A computer with Python, a modern browser and available disk space

CHAPTER 5

SYSTEM DESIGN

5.1 Architecture and input design

The browser sends same-origin requests to FastAPI. The server authenticates users, checks academic scope, validates input and accesses SQLite through SQLAlchemy. Forms show names with public codes; totals and grades are calculated on the server.

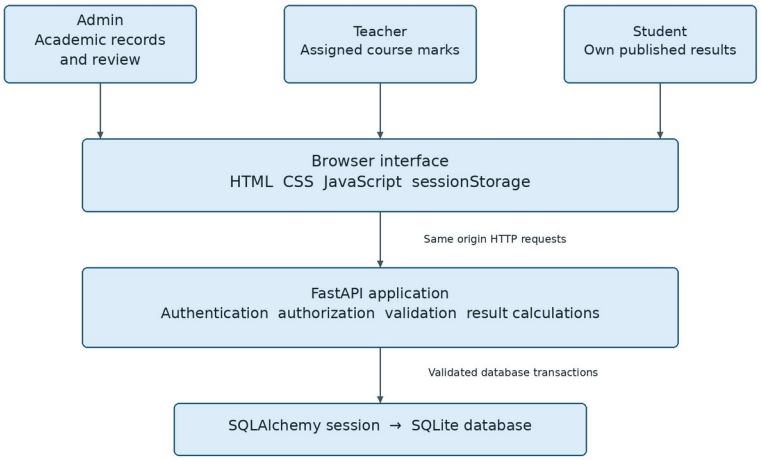


Figure 5.1 Layered architecture of the Result Processing System

5.2 Data flow and transactions

Academic setup, marks entry and published-result queries use the same database. Imports and batch changes use transactions; publication changes the existing marks record.

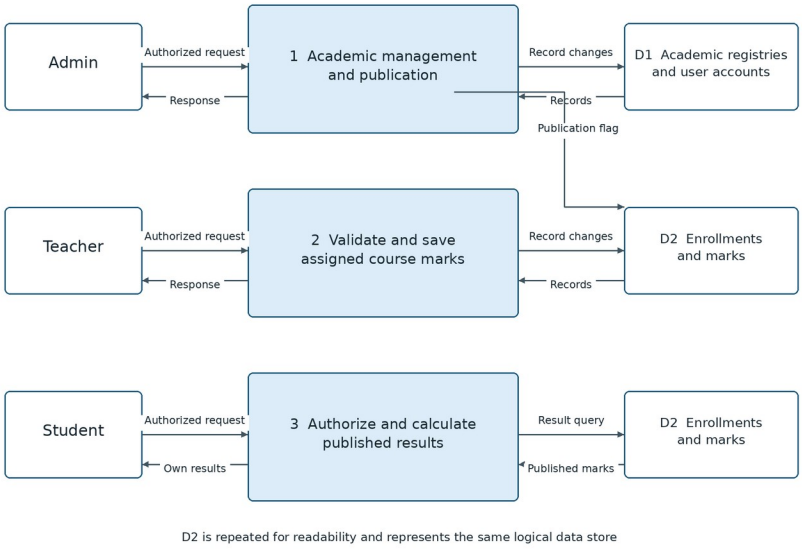


Figure 5.2 Main academic data flows and logical data stores

Use case design

5.3 Use case diagram

Administrators prepare academic data and publish reviewed results. Teachers enter marks within assigned offerings, and students read their own results. Account provisioning remains an administrative operation.

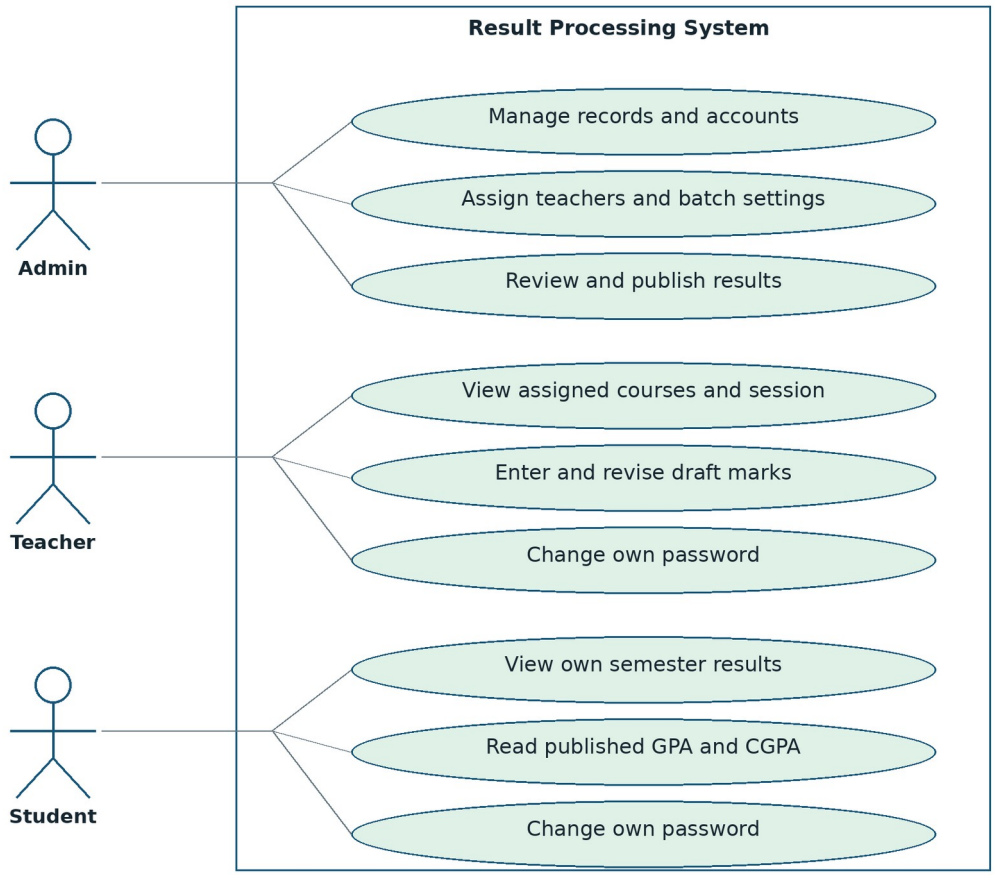


Figure 5.3 Role-specific use cases

Database design

5.4 Entity relationship diagram

Ten tables represent registries, accounts, batches, assignments, enrollments and marks. Each enrollment has at most one marks record. The user reference is an application-level link because it can identify a teacher or student; it is not a database foreign key.

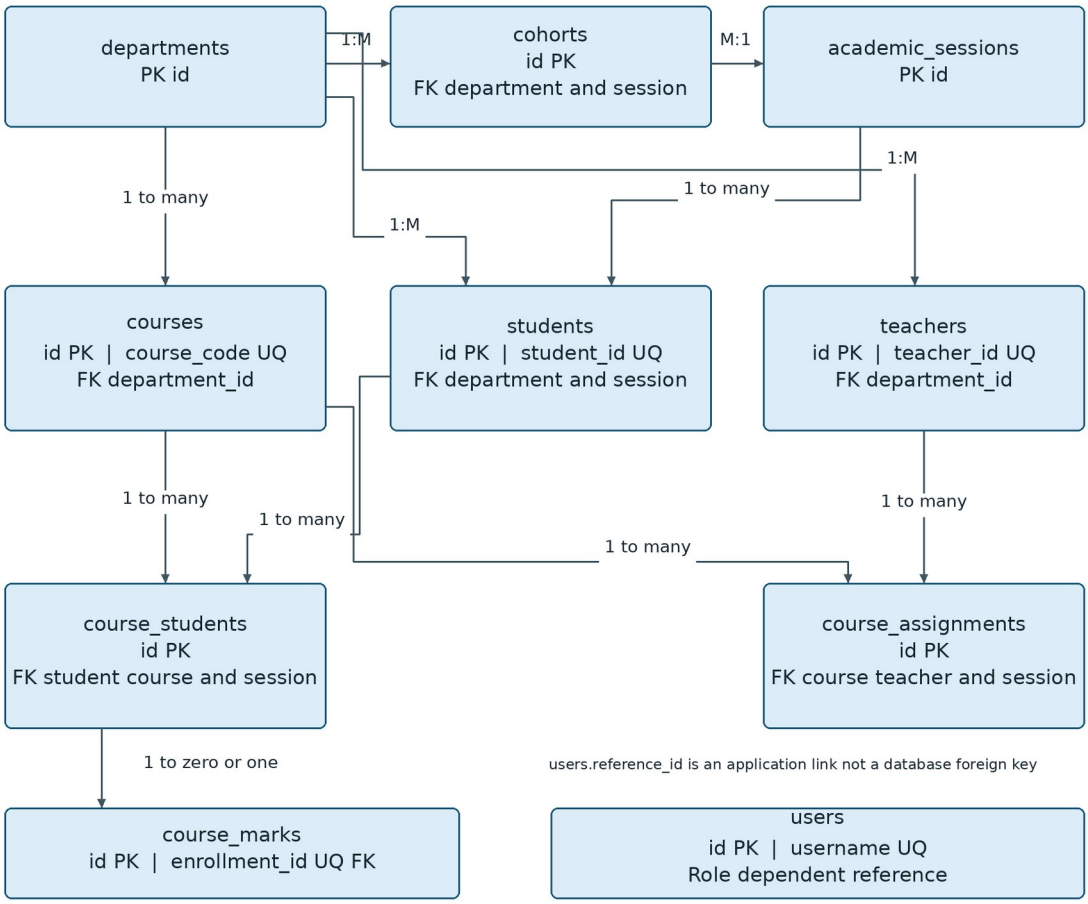


Figure 5.4 Database entities and principal relationships

Sequence and state design

5.5 Sequence diagram

A revision number links a reviewed record to its requested update. Saving, publishing or unpublishing advances the revision. Stale updates are rejected. The state sequence is no marks, draft and published; unpublishing returns a record to draft.

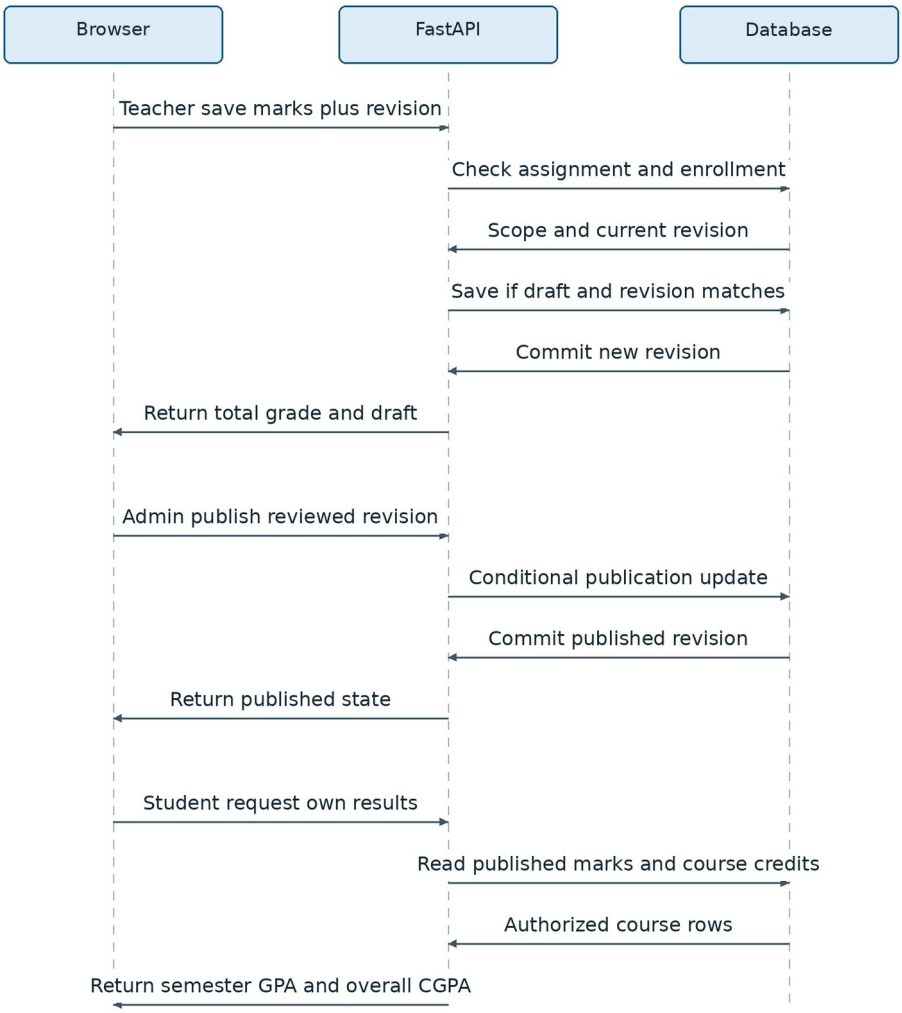


Figure 5.5 Save review publish and view sequence

CHAPTER 6

IMPLEMENTATION AND RESULTS

6.1 Implementation structure

FastAPI implements authentication, academic operations and results [5]. SQLAlchemy handles persistence; separate modules manage imports and fixed enrollment. HTML, CSS and JavaScript provide the interface.

6.2 Account and database lifecycle

Startup initializes Backend/rps.sqlite3 and synchronizes core enrollments. Administrators provision temporary passwords. First-login password changes and later resets invalidate earlier tokens.

6.3 Assessment and grade calculation

Total marks = assignment + class test + midterm + final, out of 150. The same configured weights apply to all course types. Percentage = total / 150 × 100; the mapping below is implemented in the application.

Table 6.1 Implemented grade thresholds

Minimum percentage	Minimum total out of 150	Grade	GP
80	120.00	A+	4.00
75	112.50	A	3.75
70	105.00	A−	3.50
65	97.50	B+	3.25
60	90.00	B	3.00
55	82.50	B−	2.75
50	75.00	C+	2.50
45	67.50	C	2.25
40	60.00	D	2.00
Below 40	Below 60.00	F	0.00

6.4 GPA and CGPA

GPA = sum of credit × grade point / eligible credits. CGPA uses eligible courses across all semesters. Students receive published marks only; failed published courses count as zero points, while pending courses are excluded and the semester is marked Incomplete. Decimal arithmetic is used. Two three-credit courses with GP 4.00 and 3.00 produce GPA/CGPA 3.50. Retakes are not implemented.

Login and dashboard interfaces

6.5 Login and administrative dashboard

One sign-in page routes authenticated users to their role workspace. The dashboard summarizes academic records and separates current from result-processing semester. These snapshots use fictional demonstration data.

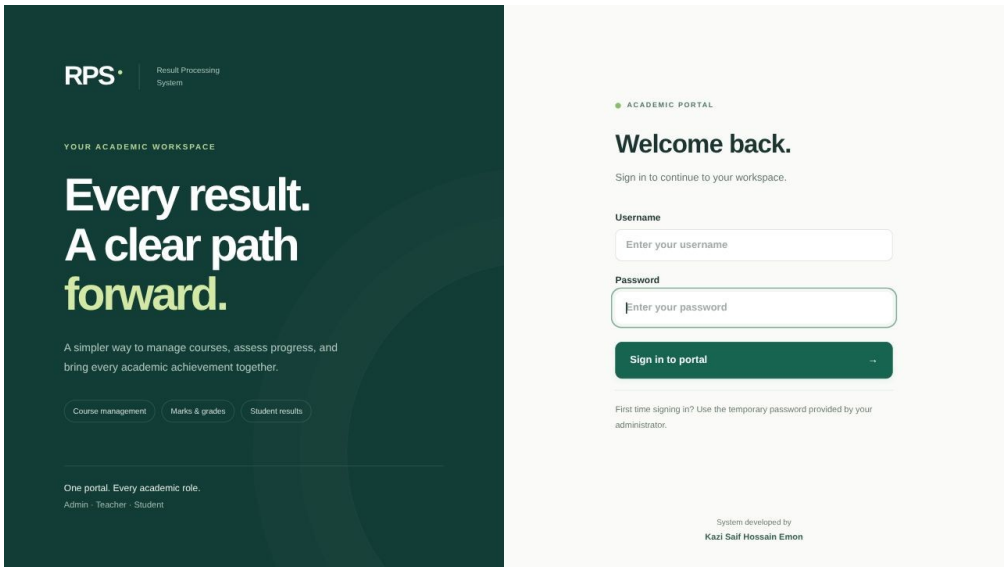


Figure 6.1 Redesigned login interface

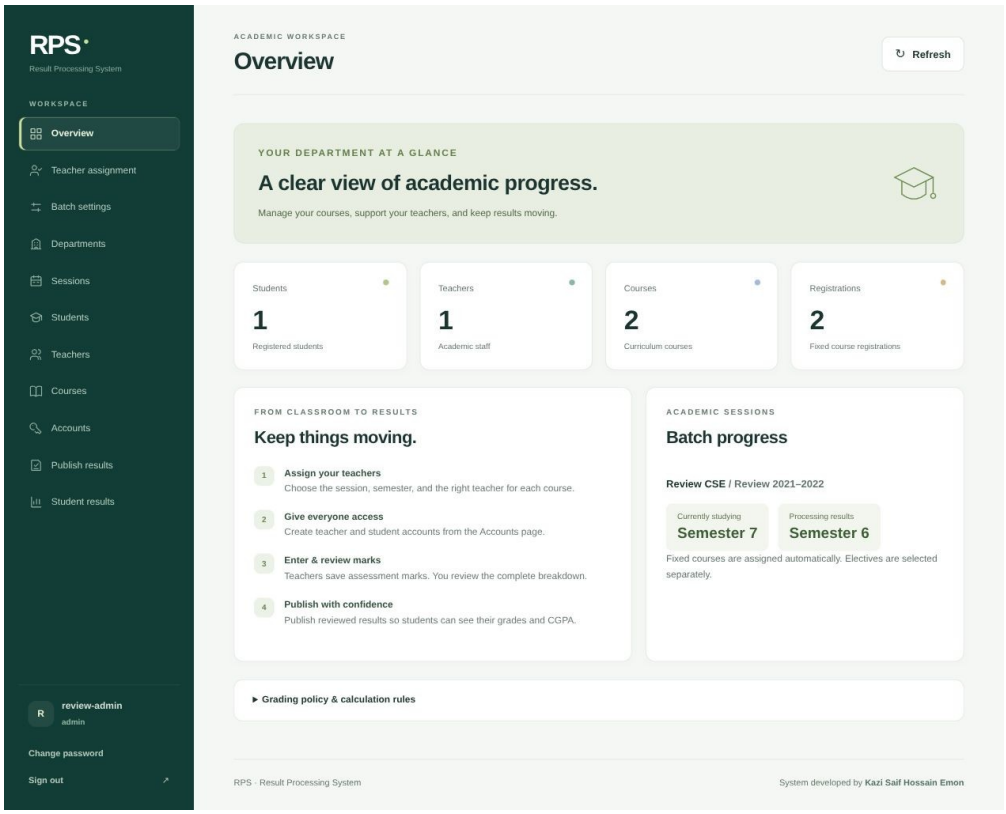


Figure 6.2 Administrative dashboard with batch progress

Marks entry and publication interfaces

6.6 Teacher entry and administrative review

Teachers select an assigned course/session, enter four bounded components and save each student separately. Unsaved input prompts confirmation. Administrators inspect the components, total and grade before publication; revision checks reject outdated saves and reviews.

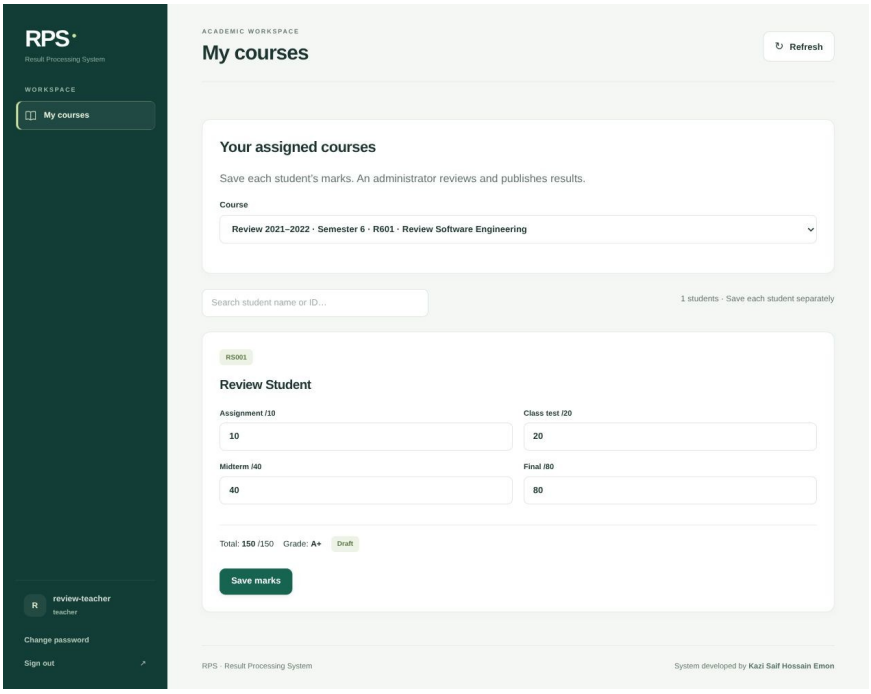


Figure 6.3 Teacher marks entry in an assigned session

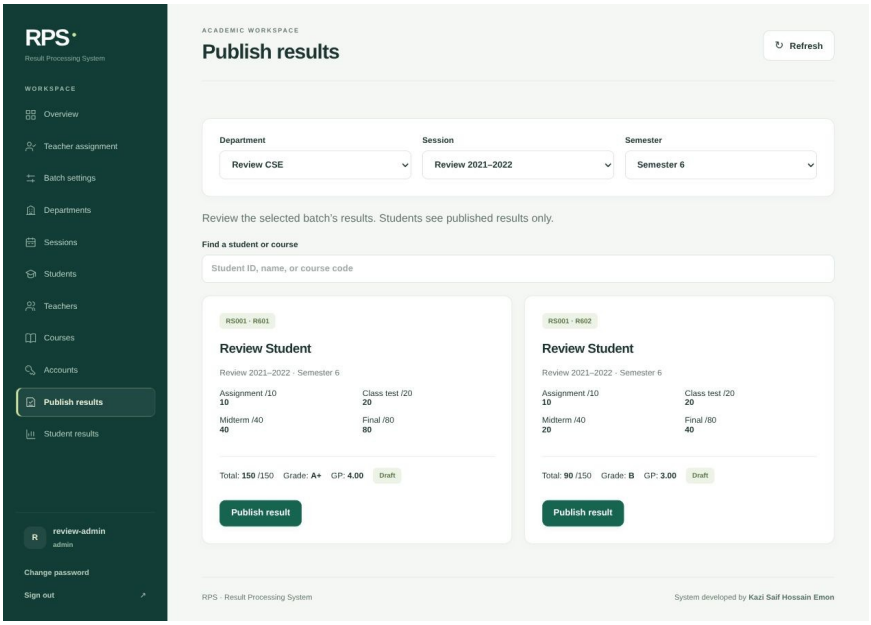


Figure 6.4 Component-level result review before publication

Student results and implementation outcome

6.7 Student results and implementation outcome

Students see their own published results, semester GPA and overall CGPA. The default semester follows batch settings. The snapshot illustrates two equal-credit courses with grade points 4.00 and 3.00, giving GPA and CGPA 3.50. This demonstration is separate from official academic records.

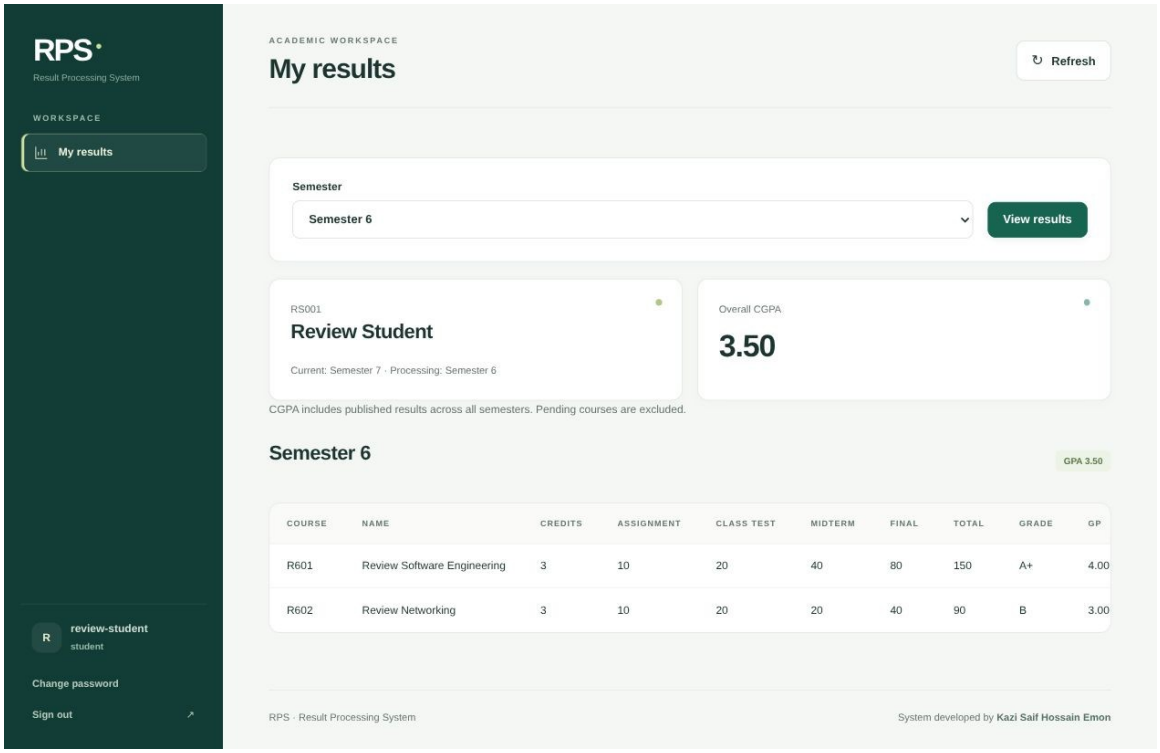


Figure 6.5 Student result view with verified GPA and CGPA

6.8 Operational boundaries

The prototype supports the local academic workflow. Official grading approval, retake rules, audit history and public hosting remain future work. Interface examples do not establish production capacity or institutional acceptance.

CHAPTER 7

TESTING

7.1 Unit testing method and isolation

Python unittest checks grading functions and the marks model directly [7]. JavaScript handlers use mocked page elements, storage and fetch. Tests use isolated configuration without a live browser, server or academic database.

7.2 Unit cases and outcomes

All 18 methods passed. Cases cover grade thresholds, boundary values, pending marks and validation failures. Methods may contain multiple input subcases.

Table 7.1 Executed unit test summary

Unit group	Test methods	Passed	Failed
Grading and pending result functions	5	5	0
Marks input model	7	7	0
Frontend isolated state handlers	6	6	0
Total	18	18	0

Table 7.2 Backend unit case outcomes

ID	Unit and input	Expected outcome
U01	Every grade threshold from 120 to 60	Correct grade and grade point
U02	Total 59.99	F and GP 0
U03	Totals 0 and 150	F and A+ respectively
U04	Total 119.99	A and GP 3.75
U05	marks_info with no saved marks	Pending values and unpublished
U06	Four components at maxima	Validation succeeds
U07	Final marks 65.25	Validation succeeds
U08	Final marks 65.251	Validation error
U09	Negative or excessive component	Validation error for each field
U10	Final component omitted	Validation error
U11	NaN or infinity	Validation error
U12	Revision zero	Validation error

Unit testing snapshot

7.3 Frontend cases and execution snapshot

The six frontend cases verify confirmation, cancellation, unsaved marks, forced logout, stale unauthorized responses, readable errors and preview-dialog state [6]. The snapshot reproduces captured execution output from 3 November 2026: 12 Python methods and 6 JavaScript methods passed.

```
test_all_grade_thresholds ... ok
test_just_below_a_plus ... ok
test_just_below_pass_fails ... ok
test_missing_marks_remain_pending ... ok
test_zero_and_maximum ... ok
test_accepts_two_decimal_places ... ok
test_component_limits ... ok
test_missing_component_rejected ... ok
test_nonfinite_values_rejected ... ok
test_rejects_three_decimal_places ... ok
test_revision_must_be_positive ... ok
test_valid_full_marks ... ok

----------------------------------------------------------------------
Ran 12 tests in 0.001s

OK

Frontend state unit tests
tests 6    pass 6    fail 0
Source runner: node Frontend/tests/state.test.cjs
```

Figure 7.1 Captured Python and frontend unit test output

7.4 Interpretation

The results support only the tested functions, validation rules and state transitions. They do not establish full workflow correctness, load capacity or production security. GPA is computed inside the result endpoint and is not claimed as a separately isolated function test.

CHAPTER 8
CONCLUSIONS AND FUTURE WORK

8.1 Conclusion

RPS provides a working local path from academic setup to student result viewing. Its main contribution is the explicit connection between department, session, curriculum, teaching responsibility and publication state. Fixed core enrollment removes repetitive student-course assignment, while separate current and processing semesters support delayed result processing for a batch that has already progressed.

The implemented checks address practical failure modes: invalid marks, access to another session, stale edits, publication of an outdated review, accidental changes to credits after marks exist, and deletion of enrollments that have saved results. The user interface presents these rules through ordinary forms, selectors, student cards and confirmation controls.

8.2 Limitations

- Assessment weights are fixed at 10, 20, 40 and 80 for all course types.
- Retakes, repeated attempts and grade replacement are not modeled.
- Account credentials must be shared manually; there is no automated email or SSO.
- Official transcript generation and downloadable result sheets are not implemented.
- The local application has process-local login limits, no complete audit-history module and no production hosting configuration.

8.3 Future development

Useful extensions include institution-approved assessment policies per course type, retake handling, a verifiable result-change audit trail, student-specific elective planning, printable transcripts, notification delivery and a reviewed production deployment. These should be added only after requirements are confirmed and the existing grade calculations and access controls remain covered by tests.

8.4 Completion status

The local software workflow is implemented, and the isolated unit checks reported in Chapter 7 pass. Final academic demonstration still requires administrator-selected teaching assignments, teacher/student accounts and approved demonstration marks. Production readiness requires separate institutional and operational acceptance.

References

[1] FastAPI, “OAuth2 with Password and hashing Bearer with JWT tokens,” FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/. [Accessed: Nov. 3, 2026].

[2] SQLite, “SQLite Foreign Key Support,” SQLite Documentation. [Online]. Available: https://www.sqlite.org/foreignkeys.html. [Accessed: Nov. 3, 2026].

[3] National Institute of Standards and Technology, “Role based access control,” CSRC Glossary. [Online]. Available: https://csrc.nist.gov/glossary/term/role_based_access_control. [Accessed: Nov. 3, 2026].

[4] SQLAlchemy, “Configuring a Version Counter,” SQLAlchemy 2.1 Documentation. [Online]. Available: https://docs.sqlalchemy.org/en/21/orm/versioning.html. [Accessed: Nov. 3, 2026].

[5] K. S. H. Emon, “Result Processing System,” application source code, version 2.1.0, Institute of Science & Technology, 2026. Local project repository, reviewed Nov. 3, 2026.

[6] K. S. H. Emon, “RPS project documentation and unit test evidence,” README.md, Frontend/tests/state.test.cjs and test_rps_units.py, 2026.

[7] Python Software Foundation, “unittest — Unit testing framework,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/unittest.html. [Accessed: Nov. 3, 2026].